

國立陽明交通大學
資訊科學與工程研究所
碩士論文

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University
Master's Thesis

Compile Once, Execute Many：重複性模板實例化任務
的可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification
Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞 (Lin, Kuan-Cheng)
指導教授： 陳添福 (Chen, Tien-Fu)

中華民國 一一五年十一月
November 2026

Compile Once, Execute Many：重複性模板實例化任務的
可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification
Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞

Student： Kuan-Cheng Lin

指導教授： 陳添福 博士

Advisor： Dr. Tien-Fu Chen



November 2026
Taiwan, Republic of China

中華民國 一一五年十一月

誌 謝

(致謝待撰寫。)

林冠丞 於

國立陽明交通大學 資訊科學與工程研究所

中華民國 一一五年十一月



Compile Once, Execute Many：重複性模板實例化任務的可驗證規格編譯方法

學生：林冠丞

指導教授：陳添福 博士

國立陽明交通大學
資訊科學與工程研究所

摘 要

(中文摘要待撰寫。)

關鍵字：(待定，5-7 個)



Compile Once, Execute Many: Verifiable Specification Compilation for Recurrent Template-Instantiation Tasks

Student : Kuan-Cheng Lin

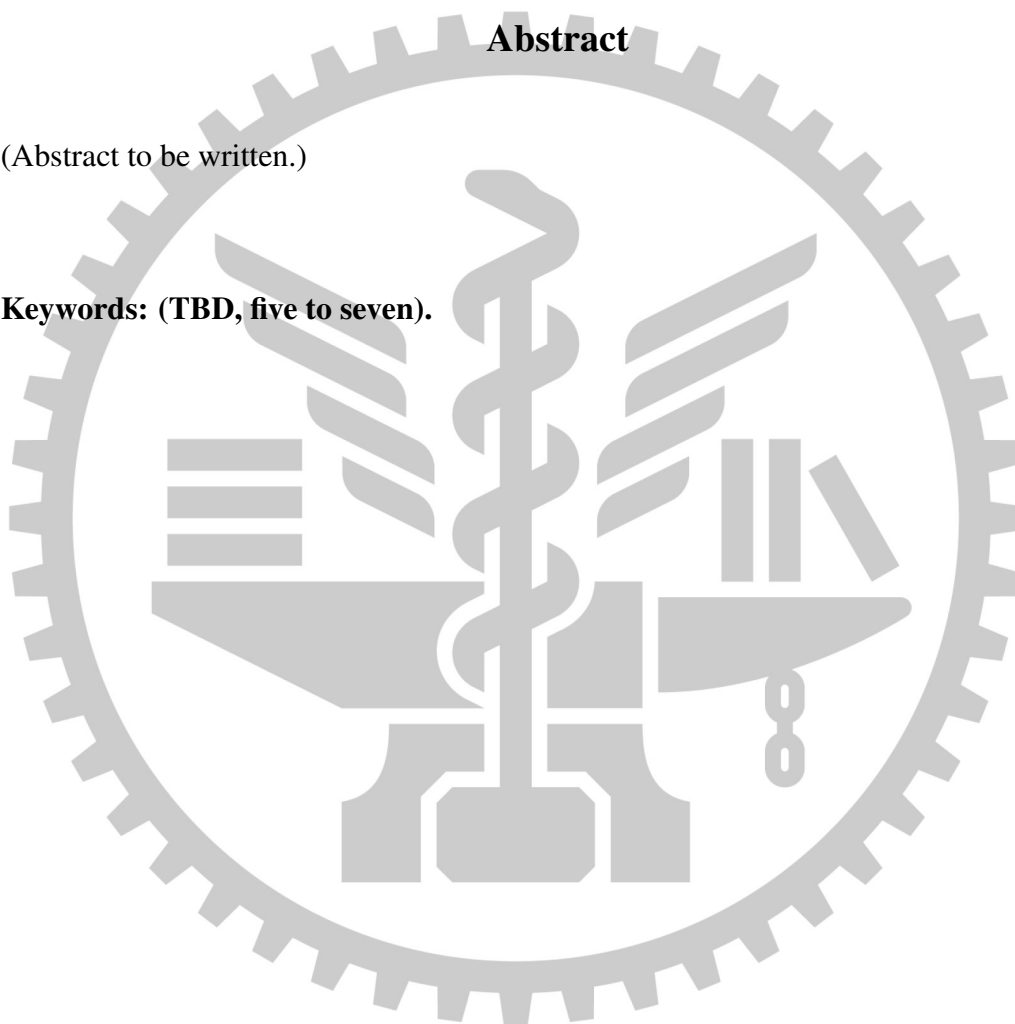
Advisor: Dr. Tien-Fu Chen

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

Abstract

(Abstract to be written.)

Keywords: (TBD, five to seven).



Contents

Chinese Abstract	i
English Abstract	ii
Contents	iii
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Motivation	1
1.2 Two Existing Approaches, Both Wrong	1
1.3 Our Position	1
1.4 Contributions	1
1.5 Claims and Where They Are Tested	1
1.6 Thesis Organization	1
2 Related Work	2
2.1 Template-Shaped Tasks	3
2.2 Document Formats and Representation Mismatch	3
2.3 Failure Modes	3
2.4 Governance Requirements	3
2.5 LLMs for Document and Engineering-Drawing Understanding	3
2.6 Document Automation, RPA, and Template Filling	3

2.7	Programming by Example and Program Synthesis	3
2.8	Tool-Making, Skill Libraries, and Amortized LLM Computation	3
2.9	Execution-Feedback Program Repair	3
2.10	Verification and Guardrails for LLM Output	3
2.11	Cost-Aware Agent Evaluation	3
3	System Model	4
3.1	Problem Formulation	4
3.1.1	A Stateful, Two-Stage Input Model	4
3.1.2	Compile-Once, Execute-Many	7
3.1.3	Required Properties of the Specification IR	9
3.1.4	Design Principle: Fail Loud, Not Silent	12
3.1.5	Scope and Applicability Conditions	12
3.2	Domain-Dependent and Domain-Independent Layers	12
3.3	Specification IR	12
3.4	Structured Dump as Grounding	12
3.5	Locator Mechanisms	12
3.5.1	Semantic Locators	12
3.5.2	Range Locators and Source-Side Aggregation	12
3.5.3	Failure Modes and What Each Mechanism Blocks	12
3.6	Target Reference and Write-Back	12
3.7	Deterministic Compiler and Zero-LLM Runtime	12
3.8	Lifecycle Management	12
3.8.1	Lineage and Versioning	12
3.8.2	Regression Suite	12

3.8.3	Drift Detection	12
3.8.4	Auditability	12
3.9	Expressiveness Boundary	12
4	Methodology	13
4.1	State Machine.....	13
4.2	Structural Gates.....	13
4.3	Two Complementary Oracles.....	13
4.4	Case-Repair Loop	13
4.5	Human in the Loop: Evidence Review	13
4.6	Coverage Is the Real Bottleneck.....	13
5	Performance Evaluation.....	14
5.1	Research Questions.....	15
5.2	Study Design.....	15
5.2.1	Why Not a Few-Shot Held-Out Split.....	15
5.2.2	Two-Level Split.....	15
5.2.3	Dataset Construction.....	15
5.3	Baselines and Fairness	15
5.4	Metrics	15
5.4.1	Asymmetric Rerun Budget	15
5.5	The Evaluator.....	15
5.6	Template Drift.....	15
5.7	Ablations.....	15
5.8	Results.....	15
5.8.1	Where the Randomness Went.....	15

5.9 Case Study: Engineering Drawings	15
5.10 Threats to Validity	15
6 Conclusions	16
6.1 Conclusion	16
6.2 Limitations and When This Method Does Not Apply	16
6.3 Future Work	16
References	17



List of Figures



List of Tables



Chapter 1. Introduction

1.1 Motivation

1.2 Two Existing Approaches, Both Wrong

1.3 Our Position

1.4 Contributions

1.5 Claims and Where They Are Tested

1.6 Thesis Organization



Chapter 2. Related Work

2.1 Template-Shaped Tasks

2.2 Document Formats and Representation Mismatch

2.3 Failure Modes

2.4 Governance Requirements

2.5 LLMs for Document and Engineering-Drawing Understanding

2.6 Document Automation, RPA, and Template Filling

2.7 Programming by Example and Program Synthesis

2.8 Tool-Making, Skill Libraries, and Amortized LLM Computation

2.9 Execution-Feedback Program Repair

2.10 Verification and Guardrails for LLM Output

Chapter 3. System Model

3.1 Problem Formulation

3.1.1 A Stateful, Two-Stage Input Model

The abbreviation $\mathcal{A} : T \rightarrow S$ is potentially misleading because the system does not consume all experimental material in one call, and derivation is neither pure nor deterministic. Let d denote one derivation dataset in template lineage ℓ . We split the available material into a derivation layer and a later regression layer,

$$T = (T_{\text{der},d}^{(n)}, T_{\text{reg},\ell}), \quad (3.1)$$

$$T_{\text{der},d}^{(n)} = (d, \ell, B_d, Y_d^?, \{X_{dj}\}_{j=1}^{N_d}, R_d^?, U_d^?, Q_d, P, \Gamma, \Theta, H_d^{(<n)}), \quad (3.2)$$

$$T_{\text{reg},\ell} = \mathcal{R}_\ell = \{c_i = (I_i, \{X_{ij}\}_{j=1}^{N_i}, u_i, \pi_i, Y_i^?)\}_{i=1}^{K_\ell}. \quad (3.3)$$

Here, B_d is exactly one baseline template; $Y_d^?$ is an optional expected output; $\{X_{dj}\}_{j=1}^{N_d}$ is a possibly empty collection of source artifacts; $R_d^?$ is an optional operational-rule artifact; and $U_d^?$ is an optional inline source-data override. When present, the override bypasses the locally parsed artifact dictionary and, on the agent path, takes precedence over extracted keys. Q_d groups the inspection level, force-refresh choice, content-addressed dump, cache state, CAD preflight and health observations, locator verification, and deterministic validation/repair feedback. The derivation environment is also an input: P denotes the prompt files, Γ the agent's

tool and skill definitions, and Θ the model, gateway, and MCP configuration. Thus, changing a prompt, an allowed tool, or the selected model changes T_{der} even when all customer artifacts are byte-identical.

The term $H_d^{(<n)}$ is the pre-existing state before the n th derive. It contains the field names and rule inventories accumulated from all existing non-degraded specs derived from the same dataset. The implementation reads this state through `derivation_service.DerivationService._field_hints`, and `_previous_rule_inventory`; the degraded flag used by that selection is produced by `cad_health.degradation`. The hints are fed to the agent and the prior inventory is merged into the new spec evidence. With ω_n denoting model sampling and the observed session/tool trace, the honest derivation relation is therefore

$$S_d^{(n)} \sim \mathcal{A}(T_{\text{der},d}^{(n)}; \omega_n), \quad T_{\text{reg},\ell} \notin \text{dom}(\mathcal{A}). \quad (3.4)$$

Consequently, two derives over the same artifact bytes need not have the same context or the same output: $H_d^{(<n)}$ may have changed, and ω_n is stochastic. In this thesis, $\mathcal{A} : T \rightarrow S$ is only shorthand for the projection $\mathcal{A} : T_{\text{der}} \rightsquigarrow S$ in Equation 3.4, not a claim that $\mathcal{A}$ is a pure function.

This split follows the implemented call boundary. The dataset creation and derive paths, `api.create_dataset` and `derivation_service.DerivationService.derive`, admit one baseline, at most one expected artifact, N_d source artifacts, and at most one rules artifact. They do not read `models.DwgTestCase`. Only after S exists does the separate endpoint call `regression.RegressionRunner.run`, which selects every active test case of the spec’s lineage and executes them. K_ℓ is therefore the number of active cases at regression time; the implementation imposes no $K_\ell = 3$ limit. Each case c_i contains an input template I_i , its own source artifacts and explicit source-data override u_i , runtime parameters π_i , and an optional expected output $Y_i^?$.

The prompt, tools, and model terms above are concrete inputs rather than abstract nuisance variables. `derivation_agent.OpenCodeAgent.derive_rules` selects the `dwg-derivation` agent and `OpenCodeAgent._run` submits the task payload; the selected agent is configured by `opencode.jsonc`, its prompt is `prompts/derivation.md`, and its permitted CAD MCP tools are defined by the agent configuration and `mcp_server.server`. These files and settings must therefore be versioned with the material definition used in an experiment.

We reason about the resulting specification as

$$S = (G, L_s, L_t, F, \Pi, \mathcal{E}_S), \quad (3.5)$$

where G contains value-source and transformation semantics, L_s contains source locators, L_t contains independently derived target locators, F contains rendering rules, Π is the runtime-parameter schema, and $\mathcal{E}_S$ is derivation and validation evidence. The persisted `DwgSpec` stores these concerns across `field_map`, `rule projections`, `runtime_params`, and `evidence`; this tuple is a conceptual separation rather than a claim that each component is a distinct database column.

Information boundary. The operational-rule artifact R_d may identify source fields and state formulas, conditions, constants, and formatting requirements, but it must not contain target cell addresses, bookmark names, content-control tags, or mappings of the form `source_field` $\rightarrow$ `target_cell`. Target references belong to L_t and must be derived from the template by $\mathcal{A}$; otherwise R_d becomes an answer table and the derivation problem is leaked into its input. **Status: normative/aspirational only.** The boundary is stated but not enforced, in two independent ways. First, no admission check exists: neither `api.create_dataset`

nor `derivation_service.DerivationService._load_rules` rejects an answer-bearing rules artifact, so a hand-authored R_d containing target addresses is accepted silently. Second, the persisted projection is not structurally safe: `derivation_service.DerivationService` copies the field's target-reference slot into `DwgSpec.mapping_rules`. That slot is carrier-agnostic — its interpretation follows the target kind of the template, so it holds bookmark identities for `.docx` and cell addresses for `.xlsx`, which are exactly the two artefact classes this boundary forbids. (The field retains the legacy name `handles` from the drawing-oriented prototype; the naming is historical and does not narrow the scope.) If that projection is exported, presented as the rule table, or fed back as R_d , it exposes target answers, and $\mathcal{A}$ is then measured on its ability to copy them rather than to derive them. It must therefore remain internal, and be stripped of target references or split from the semantic rules, before the boundary may be described as a property of the system rather than a requirement on it.

3.1.2 Compile-Once, Execute-Many

The target problem class consists of recurrent template-instantiation tasks for which the output is deterministically derivable from structured source data and explicit operational rules. “Compile once” means that the expensive, stochastic derivation in Equation 3.4 produces one reusable S ; it does not mean that a single fixed output job is reused, because instance values differ. For instance input $V_i = (I_i, \{X_{ij}\}, u_i, \pi_i)$, execution deterministically instantiates and

applies the spec,

$$(J_i, P_i) = \mathcal{C}(S, V_i), \quad (3.6)$$

$$D_i = \mathcal{X}(J_i, I_i), \quad (3.7)$$

$$\text{ok}_i = \mathcal{O}_{\text{plan}}(P_i, I_i, D_i) \wedge \begin{cases} \mathcal{O}_{\text{expected}}(D_i, Y_i), & Y_i^? = Y_i, \\ \text{true}, & Y_i^? \text{ is absent.} \end{cases} \quad (3.8)$$

$\mathcal{C}$ is the deterministic per-instance compiler, P_i is its authoritative write plan, $\mathcal{X}$ is the format-specific executor, and D_i is the produced document. The problem is to derive an S once such that ok_i holds for every admissible instance, while the cost of derivation is amortized over repeated executions.

The deterministic core is implementation-backed. In `compiler.compile_cad_job`, the spec, source data, runtime parameters, and current template values are explicit inputs; failures are returned before submission, and the canonical job JSON is content-hashed. The DOCX/XLSX path shares this compiler and applies P_i through `docx_target.DocxTargetAdapter` or `xlsx_target.XlsxTargetAdapter.apply`; `document_production.document_job` records a canonical hash for the document job. Regression is also implementation-backed: `regression.RegressionRunner._run_case` compiles each case, and `regression.RegressionRunner.verify_output` uses `verification.verify_output` plus `verification.compare_expected` when Y_i exists.

This formulation deliberately localizes randomness to $\mathcal{A}$, but the current end-to-end DWG production path does not yet justify a zero-LLM execution claim. `production.ProductionControl` invokes `derivation_agent.OpenCodeAgent.repair_cad_job` after a failed gateway job validation, and `production._agent_job_preserves_inputs` excludes tar-

get locations from its business-value comparison. A repaired target can therefore be accepted and submitted within the same run. Eliminating that branch from execution, or changing it to create a reviewable draft and fail the run, is required before $\mathcal{X}$ can be described as strictly zero-LLM and deterministic.

3.1.3 Required Properties of the Specification IR

The reusable IR must satisfy the following five properties. Each status below refers to the repository state examined for this thesis, not merely to the intended design.

P1—separation of structure and instance values. S must describe reusable source/target locators, transformations, formatting, and parameter types; a particular instance supplies values through V_i . The only values permitted in S are constants explicitly prescribed by R_d , not values copied from a calibration output. **Status: implementation-backed at the representation interface.** `compiler.compile_cad_job` accepts `spec`, `source_data`, and `runtime_params` separately, while `value_layer.resolve_field_raw` evaluates the value layer at execution time; `regression.RegressionRunner._run_case` reuses the same `spec` with each case’s inputs. This interface supports reuse, although successful regression—not the type shape alone—is what detects a semantically overfit constant.

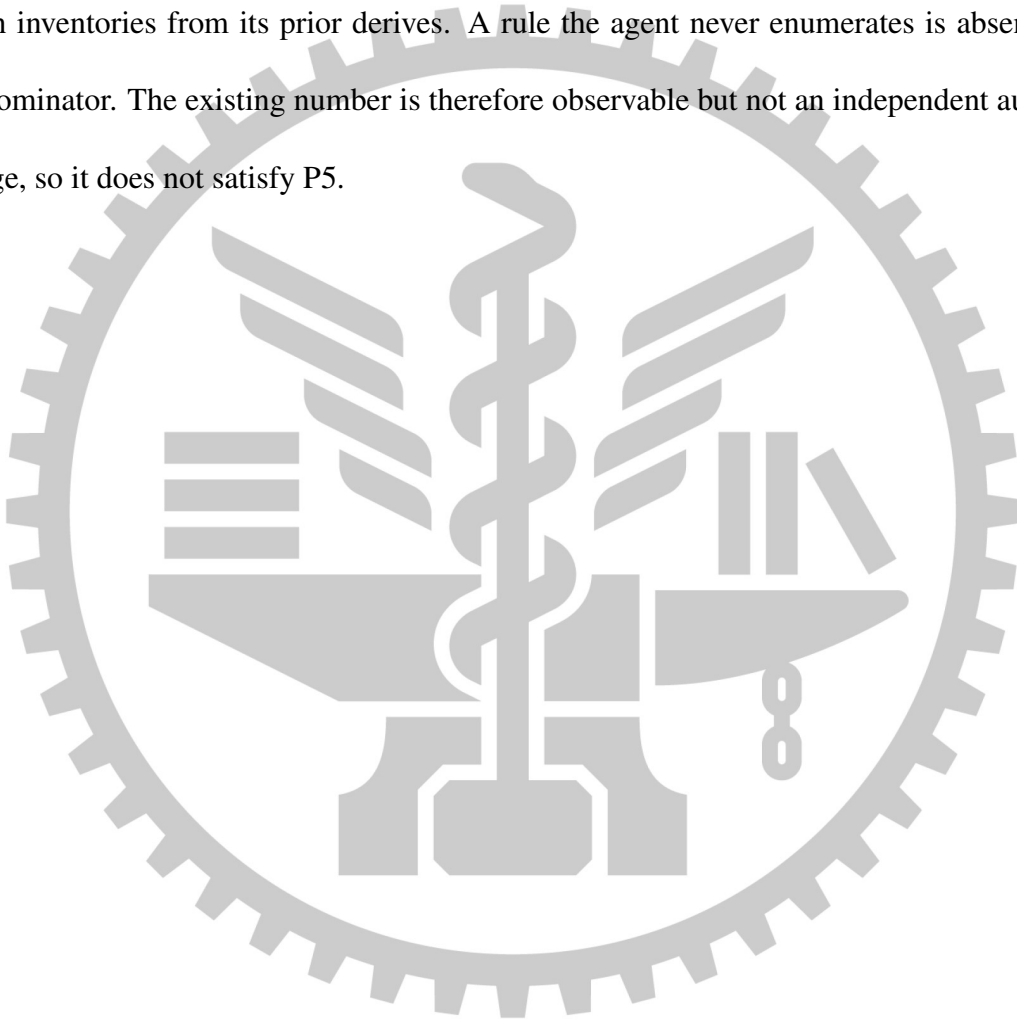
P2—explainable and verifiable locators. Every source or target locator must expose what it resolved to and must be checkable against the corresponding artifact; an unresolved or ambiguous locator is evidence, not permission to guess. **Status: implementation-backed.** `source_locator.resolve` returns the resolved sheet, cell, selection, and match count, and `source_resolution.verify_against` checks that the locator reproduces the derivation sample. On the target side, `target_ref.BaseTargetRef.resolve` and `describe` provide a common existence and explanation interface. `docx_target.DocxTargetAcquire`

and `xlsx_target.XlsxTargetAdapter.inspect` retain labels, physical locations, positional flags, and ambiguous references, while their `apply` methods return every missed reference.

P3—bounded layout resilience and fail-loud behavior. Semantic identities should survive admissible layout changes, while a change outside the declared tolerance must stop the run rather than trigger a guessed write. This includes the zero-LLM execution requirement: an execution-time model must not relocate a target. **Status: normative/aspirational only.** There is substantial partial support: source locators resolve by sheet/header/section semantics; `docx_target.DocxTargetAdapter` distinguishes stable content-control/bookmark identities from positional table cells; `xlsx_target.XlsxTargetAdapter` distinguishes stable defined names from positional coordinates; and both reject duplicate or missing references. However, the DWG agent-repair branch in `production.ProductionController` described above can alter target locations and continue the same execution. Therefore the end-to-end fail-loud property is not presently enforced.

P4—diffability, versioning, and lineage. The IR and its evidence must support reviewable changes, immutable publication, and regression over a template lineage. **Status: implementation-backed for the currently produced handle-backed field-map shape.** `models.DwgSpec` carries a lineage-scoped unique version and template fingerprint; `drift.spec_field_drift` reports added, removed, and attribute-level field changes; `models.DwgProductionVersion.save` prevents mutation of the published spec/hash identity; and `regression.RegressionRunner.run` persists results for all active cases of the lineage. The drift function reads `target.handles` rather than the generic `targets` representation; diffability for a future spec that uses only generic target refs remains normative until that path is implemented.

P5—independently auditable rule coverage. *S* must enumerate which rules and fields it claims to cover, and the denominator must come from the rule artifact independently of the model being scored. **Status: normative/aspirational only.** The repository implements book-keeping in `rule_coverage.merge_inventory` and `score_coverage`, and persists the result from `derivation_service.DerivationService.derive`. Nevertheless, the current `rule_inventory` is generated by the same agent that produces *S* and is unioned with inventories from its prior derives. A rule the agent never enumerates is absent from the denominator. The existing number is therefore observable but not an independent audit of coverage, so it does not satisfy P5.



3.1.4 Design Principle: Fail Loud, Not Silent

3.1.5 Scope and Applicability Conditions

3.2 Domain-Dependent and Domain-Independent Layers

3.3 Specification IR

3.4 Structured Dump as Grounding

3.5 Locator Mechanisms

3.5.1 Semantic Locators

3.5.2 Range Locators and Source-Side Aggregation

3.5.3 Failure Modes and What Each Mechanism Blocks

3.6 Target Reference and Write-Back

3.7 Deterministic Compiler and Zero-LLM Runtime

3.8 Lifecycle Management

3.8.1 Lineage and Versioning

3.8.2 Regression Suite

3.8.3 Drift Detection

3.8.4 Auditability

Chapter 4. Methodology

4.1 State Machine

4.2 Structural Gates

4.3 Two Complementary Oracles

4.4 Case-Repair Loop

4.5 Human in the Loop: Evidence Review

4.6 Coverage Is the Real Bottleneck





Chapter 5. Performance Evaluation

5.1 Research Questions

5.2 Study Design

5.2.1 Why Not a Few-Shot Held-Out Split

5.2.2 Two-Level Split

5.2.3 Dataset Construction

5.3 Baselines and Fairness

5.4 Metrics

5.4.1 Asymmetric Rerun Budget

5.5 The Evaluator

5.6 Template Drift

5.7 Ablations

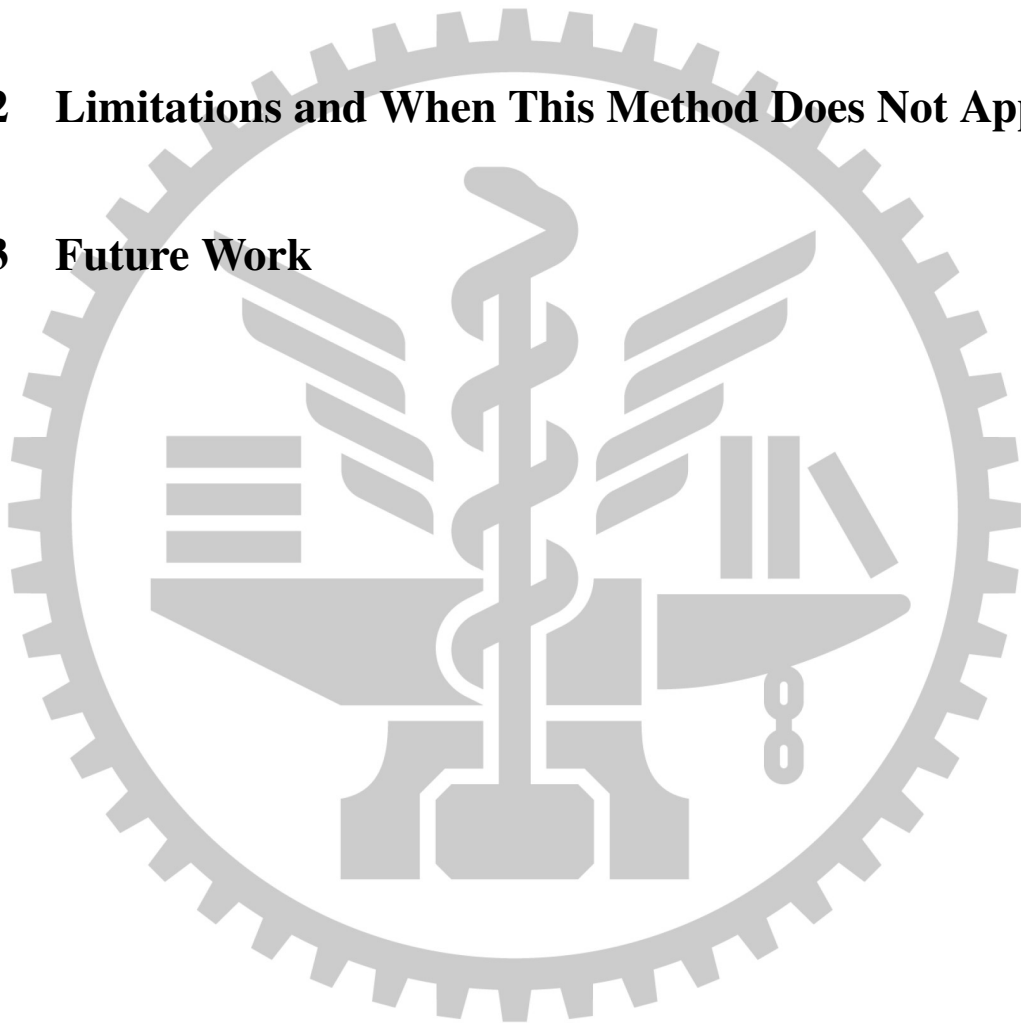
5.8 Results

Chapter 6. Conclusions

6.1 Conclusion

6.2 Limitations and When This Method Does Not Apply

6.3 Future Work



References

